

Crime Network Intelligence System (CNIS)

AI-powered link analysis for investigators — reference architecture + runnable demo

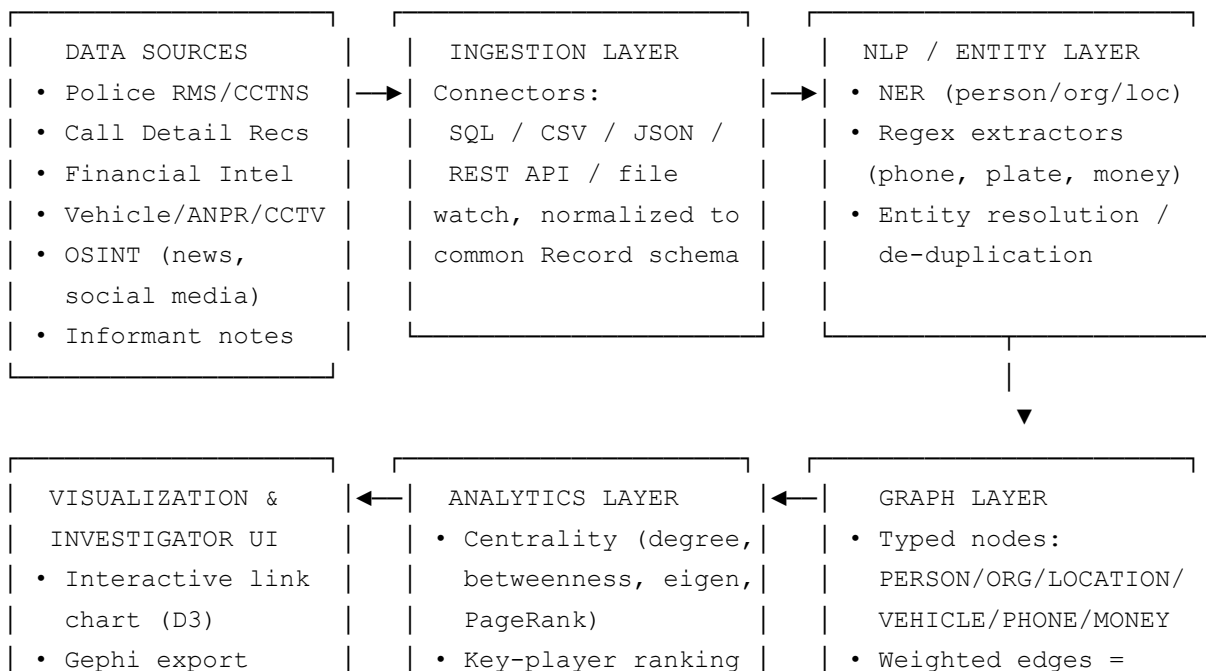
This package contains a working reference implementation of the system described in the requirements: it ingests multi-source case data, extracts entities, builds a relationship graph, ranks key influencers, flags suspicious patterns, and produces investigator-facing visual outputs.

Run it with:

```
pip install -r requirements.txt
cd src
python3 pipeline.py
```

Outputs land in `output/`: an interactive link-chart (`network_graph.html`), a Gephi-compatible graph (`network.gexf`), a ranked-players chart (`top_players.png`), and a full JSON intelligence report.

1. System Architecture



• PDF/report export	• Community detection	corroborating record
• Alerts/dashboard	• Anomaly detection	count
	(burst, structuring	• Backed by Neo4j/
	outlier models)	Neptune in production

2. Process Flow (mapped to your requirements)

#	Requirement	How it's implemented
1	Collect/process multi-source data	<code>ingestion.py</code> — pluggable connectors (SQL, CSV, JSON, REST API), all normalized into one <code>Record</code> schema and de-duplicated
2	Extract entities (people, locations, vehicles, phones, orgs)	<code>entity_extraction.py</code> — NER backend interface + regex extractors for phone numbers, vehicle plates, currency amounts; gazetteer/NER for names, orgs, locations
3	Build relationship maps	<code>graph_builder.py</code> — co-occurrence graph, edge weight = number of independent corroborating records
4	Identify key/influential individuals	<code>network_analysis.py</code> — degree, betweenness, eigenvector centrality + PageRank blended into a composite influence score
5	Detect suspicious patterns	<code>anomaly_detection.py</code> — burst-activity detection, structuring (money-laundering) language detection, new-entity-spike detection, Isolation Forest statistical outliers
6	Visual & analytical insights for investigators	<code>visualize.py</code> — interactive force-directed HTML link chart, Gephi <code>.gexf</code> export, ranked bar chart, full JSON report

3. Data flow, step by step

- Ingestion** — Each source (RMS database, telecom CDR dump, FIU/bank alerts, ANPR/vehicle feeds, informant notes, OSINT) is read through its own connector but yields the same `Record(record_id, source, date, text, structured)` object, so downstream code never has to special-case a source.
- Entity extraction** — Every record's free text is run through NER to pull out `PERSON`, `ORG`, `LOCATION` mentions, and through regex for `PHONE`, `VEHICLE` plate, and `MONEY`

amounts. Phone numbers are normalized (last 10 digits) so the same number written in different formats across sources resolves to one entity — a basic form of entity resolution. Production systems should extend this with fuzzy name matching (Jaro-Winkler / embeddings) to merge spelling variants and aliases.

3. **Relationship graph** — Any two entities appearing in the same record become a candidate edge ("associated via record X"). Repeated co-occurrence across multiple independent records increases edge weight, which is a simple but effective proxy for relationship strength/confidence.

4. **Network analysis:**

- **Degree centrality** → who has the most direct contacts (hubs/organizers).
- **Betweenness centrality** → who bridges otherwise-separate clusters (couriers, brokers — high-value disruption targets).
- **Eigenvector centrality** → who is connected to other well-connected people (often leadership).
- **PageRank** → a robust blended influence score.
- **Louvain community detection** → surfaces sub-crews/cells inside a larger network automatically.

5. **Anomaly/pattern detection** — Rule-based detectors catch known signatures (burst calling, transaction structuring, a new player entering fully-linked into an existing cluster); an Isolation Forest model on centrality features catches outlier patterns the rules don't anticipate.

6. **Visualization & reporting** — An interactive, filterable link chart (open in any browser), a Gephi-compatible export for deeper manual analysis, a ranked key-player chart, and a structured JSON intelligence report that a case-management or BI dashboard can consume directly.

4. Taking this to production

Component	Demo (this repo)	Production upgrade
NER	Regex + gazetteer	Fine-tuned transformer NER (spaCy/HuggingFace) trained on police-report language; multilingual support
Entity resolution	Exact/normalized match	Fuzzy matching, alias/nickname resolution, cross-source identity resolution (e.g. Splink, Zingg)

Graph storage	In-memory NetworkX	Neo4j / Amazon Neptune / TigerGraph for scale, live Cypher/Gremlin queries, role-based access control
Ingestion	File/manual connectors	Scheduled ETL (Airflow/Prefect), streaming ingestion (Kafka) for CDR/ANPR feeds
Anomaly detection	Rule-based + Isolation Forest	Temporal graph neural networks, sequence pattern mining, supervised models trained on closed-case outcomes
Access & audit	None	Role-based access control, full query audit logging, chain-of-custody tracking — essential for evidentiary use
UI	Static HTML export	Full investigator dashboard (search, case linking, alerting, report export to Word/PDF)

5. Governance note

A system like this touches personal data and can materially affect people's liberty, so production deployment should be paired with clear legal authority for each data source, defined retention/purge rules, human review before any automated "flag" drives an investigative or enforcement action, and an audit trail of who queried what and why.